

NSS COLLEGE APPLICATION

Feature-Wise Implementation Specification

B.Tech College — National Service Scheme (NSS)

Document Item	Value
Project	NSS College Management Application
Document Type	Feature-wise functional + frontend + backend implementation specification
Target	B.Tech / Engineering College
Primary Roles	Volunteer, Student Leader, Programme Officer, Coordinator, Administrator
API Style	Versioned REST API
Database	PostgreSQL
Document Version	1.0

Purpose: Provide an implementation-ready specification for every major NSS application feature, including user workflow, UI, backend behavior, data model, APIs, validation, authorization, edge cases and tests.

1. Authentication & Session Management

Purpose & Functional Requirements

- Allow authorized college users to sign in and establish an application session.
- Support logout and session expiry.
- Provide current-user identity and role information to the application.
- Prevent brute-force authentication through rate limiting.
- Record security-relevant authentication events.

API Contract

Operation	Endpoint / Mechanism	Purpose
Login	POST /api/v1/auth/login	Authenticate credentials and establish session/token.
Current User	GET /api/v1/auth/me	Return authenticated user profile and effective roles.
Logout	POST /api/v1/auth/logout	Invalidate the active session where applicable.
Refresh	POST /api/v1/auth/refresh	Rotate/refresh session credentials if token architecture uses it.

Data Components

users, sessions or refresh_tokens, roles, user_roles, audit_logs

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

2. User & Profile Management

Purpose & Functional Requirements

- Maintain the institutional user identity.
- Allow users to view their own profile.
- Allow authorized administrators to manage account status.
- Keep identity fields separate from NSS-specific volunteer data.

API Contract

Operation	Endpoint / Mechanism	Purpose
View Profile	GET /api/v1/users/me	Return current user profile.
Update Profile	PATCH /api/v1/users/me	Update fields allowed by policy.
Admin Users	GET /api/v1/admin/users	Paginated administrative user listing.
Status	PATCH /api/v1/admin/users/:id/status	Activate/deactivate/archive according to policy.

Data Components

users, roles, user_roles

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

3. Roles & Permissions

Purpose & Functional Requirements

- Implement server-side RBAC.
- Scope permissions by institution/unit where required.

- Prevent privilege escalation through client-controlled role data.
- Audit sensitive permission changes.

API Contract

Operation	Endpoint / Mechanism	Purpose
Roles	GET /api/v1/roles	List role definitions.
Assign Role	POST /api/v1/admin/users/:id/roles	Assign an authorized role.
Remove Role	DELETE /api/v1/admin/users/:id/roles/:roleId	Remove role when permitted.

Data Components

roles, permissions, user_roles, audit_logs

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

4. Volunteer Registration & Management

Purpose & Functional Requirements

- Create and maintain NSS volunteer profiles.
- Capture academic and NSS participation information.
- Associate volunteers with NSS units.
- Support approval/activation workflows.
- Provide searchable and paginated volunteer records.

API Contract

Operation	Endpoint / Mechanism	Purpose
List	GET /api/v1/volunteers	Search/filter/paginate volunteers.
Create	POST /api/v1/volunteers	Create volunteer record.

Details	GET /api/v1/volunteers/:id	Return authorized volunteer details.
Update	PATCH /api/v1/volunteers/:id	Update permitted fields.
Membership	POST /api/v1/volunteers/:id/unit-memberships	Assign unit membership.

Data Components

volunteers, unit_memberships, nss_units, users

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

5. NSS Unit Management

Purpose & Functional Requirements

- Create and maintain NSS units.
- Assign Programme Officers and permitted student leaders.
- View unit membership and participation summaries.
- Restrict unit-scoped operations to authorized staff.

API Contract

Operation	Endpoint / Mechanism	Purpose
List Units	GET /api/v1/units	List units.
Create Unit	POST /api/v1/units	Create unit.
Unit Details	GET /api/v1/units/:id	Return unit information.
Update Unit	PATCH /api/v1/units/:id	Update authorized fields.
Members	GET /api/v1/units/:id/members	Paginated unit members.

Data Components

nss_units, unit_memberships, users

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

6. Event Management

Purpose & Functional Requirements

- Create, publish and manage NSS events.
- Support event lifecycle states.
- Define venue, date/time, unit, capacity and eligibility.
- Prevent invalid state transitions.
- Expose only events visible to the requesting user's scope.

API Contract

Operation	Endpoint / Mechanism	Purpose
List	GET /api/v1/events	Filter by date, unit, type and status.
Details	GET /api/v1/events/:id	Return event details.
Create	POST /api/v1/events	Create event.
Update	PATCH /api/v1/events/:id	Modify authorized event.
Publish	POST /api/v1/events/:id/publish	Publish event.
Cancel	POST /api/v1/events/:id/cancel	Cancel with reason where required.

Data Components

events, nss_units, users

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

7. Event Registration

Purpose & Functional Requirements

- Allow eligible volunteers to register for published events.
- Prevent duplicate registration.
- Enforce capacity transactionally.
- Support cancellation before the configured cutoff.
- Expose registration status.

API Contract

Operation	Endpoint / Mechanism	Purpose
Register	POST /api/v1/events/:id/registrations	Register current volunteer.
Cancel	DELETE /api/v1/events/:id/registrations	Cancel own registration if permitted.
My Registration	GET /api/v1/events/:id/registrations/me	Return current registration state.
Participants	GET /api/v1/events/:id/registrations	Authorized participant list.

Data Components

event_registrations, events, volunteers

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.

- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

8. Attendance / QR Check-in

Purpose & Functional Requirements

- Create time-bounded attendance sessions.
- Generate short-lived server-controlled attendance tokens where QR is used.
- Bind check-in to authenticated volunteer identity.
- Reject expired, invalid, duplicate or unauthorized check-ins.
- Support authorized correction workflow.

API Contract

Operation	Endpoint / Mechanism	Purpose
Open Session	POST /api/v1/events/:id/attendance-sessions	Create attendance session.
Session State	GET /api/v1/attendance-sessions/:id	Return session state.
Check-in	POST /api/v1/attendance/check-ins	Validate token/session and record attendance.
Attendance	GET /api/v1/events/:id/attendance	Authorized attendance list.
Correction	POST /api/v1/attendance/:id/corrections	Request/perform authorized correction.

Data Components

attendance_sessions, attendance_records, attendance_corrections, events, volunteers

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

9. Service Hours

Purpose & Functional Requirements

- Maintain an authoritative ledger of verified service hours.
- Derive hours from approved attendance/activity sources.
- Allow controlled manual adjustments with reasons and authorization.
- Provide volunteer totals and event-level breakdowns.

API Contract

Operation	Endpoint / Mechanism	Purpose
My Hours	GET /api/v1/volunteers/me/service-hours	Return verified hours.
Volunteer Hours	GET /api/v1/volunteers/:id/service-hours	Authorized detailed history.
Adjustment	POST /api/v1/service-hours/adjustments	Create authorized adjustment.
Approve	POST /api/v1/service-hours/adjustments/:id/approve	Approve/reject adjustment.

Data Components

service_hour_entries, attendance_records, events, audit_logs

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.

- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

10. Activity Reports

Purpose & Functional Requirements

- Allow authorized users to create, save and submit event activity reports.
- Support report review and approval.
- Store structured information and document references.
- Keep submitted/approved records auditable.

API Contract

Operation	Endpoint / Mechanism	Purpose
Create	POST /api/v1/activity-reports	Create draft report.
Update	PATCH /api/v1/activity-reports/:id	Edit permitted draft.
Submit	POST /api/v1/activity-reports/:id/submit	Submit for review.
Review	POST /api/v1/activity-reports/:id/review	Approve/reject according to role.
Details	GET /api/v1/activity-reports/:id	Return report.

Data Components

activity_reports, events, documents, users, audit_logs

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

11. Announcements

Purpose & Functional Requirements

- Publish notices to volunteers or selected NSS units/roles.
- Support expiry.
- Expose read/unread state where notifications are integrated.
- Restrict publishing to authorized roles.

API Contract

Operation	Endpoint / Mechanism	Purpose
List	GET /api/v1/announcements	Return visible announcements.
Create	POST /api/v1/announcements	Create draft/published announcement.
Update	PATCH /api/v1/announcements/:id	Edit according to lifecycle.
Publish	POST /api/v1/announcements/:id/publish	Publish announcement.

Data Components

announcements, users, nss_units

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

12. Notifications

Purpose & Functional Requirements

- Create in-app notifications from configured events.
- Track unread/read status.
- Use background jobs for high-volume external delivery.
- Avoid duplicate notification creation for retryable jobs.

API Contract

Operation	Endpoint / Mechanism	Purpose
List	GET /api/v1/notifications	Paginated notifications.
Read	POST /api/v1/notifications/:id/read	Mark notification read.
Read All	POST /api/v1/notifications/read-all	Mark visible notifications read.

Data Components

notifications, users

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

13. Achievements

Purpose & Functional Requirements

- Store approved awards, recognitions and leadership achievements.
- Allow authorized creation/review.
- Prevent unauthorized modification of approved records.
- Expose achievement history to volunteers.

API Contract

Operation	Endpoint / Mechanism	Purpose
List	GET /api/v1/volunteers/me/achievements	Current volunteer achievements.
Create	POST /api/v1/achievements	Authorized creation.
Review	POST /api/v1/achievements/:id/review	Approve/reject.
Details	GET /api/v1/achievements/:id	Return achievement.

Data Components

achievements, volunteers, documents, audit_logs

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

14. Certificates

Purpose & Functional Requirements

- Track certificate eligibility and issuance.
- Generate or reference institution-approved certificate artifacts.
- Restrict access to authorized certificate records.
- Maintain issuance history.

API Contract

Operation	Endpoint / Mechanism	Purpose
List	GET /api/v1/certificates	Visible certificates.
Eligibility	GET /api/v1/volunteers/:id/certificate-eligibility	Authorized eligibility result.

Issue	POST /api/v1/certificates	Issue certificate.
Download	GET /api/v1/certificates/:id/download	Authorized artifact access.

Data Components

certificates, volunteers, documents, audit_logs

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

15. Documents & Media

Purpose & Functional Requirements

- Handle controlled upload metadata and object storage references.
- Validate file type and size server-side.
- Authorize every download.
- Keep storage identifiers non-guessable.
- Avoid storing unnecessary sensitive information.

API Contract

Operation	Endpoint / Mechanism	Purpose
Upload	POST /api/v1/documents	Upload or register approved document.
Metadata	GET /api/v1/documents/:id	Return authorized metadata.
Download	GET /api/v1/documents/:id/download	Stream authorized document.
Delete	DELETE /api/v1/documents/:id	Delete/archive if policy permits.

Data Components

documents, activity_reports, achievements, certificates

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

16. Dashboards & Analytics

Purpose & Functional Requirements

- Provide role-specific aggregate statistics.
- Keep dashboard queries permission-scoped.
- Use optimized aggregate queries.
- Cache only where staleness is acceptable.

API Contract

Operation	Endpoint / Mechanism	Purpose
Volunteer Dashboard	GET /api/v1/dashboard/volunteer	Upcoming events, hours, attendance, notices.
Coordinator Dashboard	GET /api/v1/dashboard/coordinator	Unit/event/attendance/report metrics.
Admin Dashboard	GET /api/v1/dashboard/admin	Institution-level metrics.

Data Components

events, attendance_records, service_hour_entries, volunteers, reports

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.

- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

17. Search, Filtering & Pagination

Purpose & Functional Requirements

- Support server-side filtering for large datasets.
- Use bounded page sizes.
- Validate sorting fields against an allowlist.
- Prevent expensive unbounded queries.

API Contract

Operation	Endpoint / Mechanism	Purpose
Events	GET /api/v1/events?page=1&pageSize=20&status=published	Paginated event query.
Volunteers	GET /api/v1/volunteers?page=1&pageSize=20	Paginated volunteer query.
Reports	GET /api/v1/reports?from=...&to=...	Filtered report query.

Data Components

Relevant feature tables plus indexes

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.

- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

18. Reports & Exports

Purpose & Functional Requirements

- Generate attendance, participation, service-hour and unit reports.
- Apply permission-aware filters.
- Queue large exports asynchronously.
- Provide secure, expiring download access.

API Contract

Operation	Endpoint / Mechanism	Purpose
Create Job	POST /api/v1/reports	Queue report.
Status	GET /api/v1/reports/:id	Return queued/processing/ready/failed.
Download	GET /api/v1/reports/:id/download	Authorized report download.

Data Components

reports, report_jobs, audit_logs

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.

- Boundary conditions are covered by automated tests.

19. Audit Logs

Purpose & Functional Requirements

- Record security-sensitive and administrative actions.
- Capture actor, target, action, timestamp and relevant metadata.
- Do not log passwords, access tokens or unnecessary sensitive payloads.
- Restrict audit access.

API Contract

Operation	Endpoint / Mechanism	Purpose
Admin Search	GET /api/v1/admin/audit	Authorized audit query.
Details	GET /api/v1/admin/audit/:id	Audit record details.

Data Components

audit_logs

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

20. Administration & Configuration

Purpose & Functional Requirements

- Provide institution-level configuration to authorized administrators.
- Manage units, roles, account state and controlled reference data.
- Audit configuration changes.

- Separate configuration from hard-coded application logic where appropriate.

API Contract

Operation	Endpoint / Mechanism	Purpose
Units	/api/v1/admin/units	Unit administration.
Users	/api/v1/admin/users	User administration.
Roles	/api/v1/admin/roles	Role/permission administration.
Settings	/api/v1/admin/settings	Approved application configuration.

Data Components

users, roles, permissions, nss_units, audit_logs

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

21. Backend Health & Operations

Purpose & Functional Requirements

- Expose liveness/readiness checks.
- Provide structured logs and correlation IDs.
- Monitor API latency, errors, database health and job queues.
- Keep operational endpoints protected as appropriate.

API Contract

Operation	Endpoint / Mechanism	Purpose
Liveness	GET /health/live	Process health.
Readiness	GET /health/ready	Dependency readiness.

Data Components

Operational infrastructure; no business tables required

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

22. Security & Abuse Protection

Purpose & Functional Requirements

- Validate every external input.
- Enforce authentication and authorization server-side.
- Rate-limit abuse-sensitive endpoints.
- Use safe database query patterns.
- Apply secure CORS and transport configuration.
- Run dependency/security scans in CI.

API Contract

Operation	Endpoint / Mechanism	Purpose
Rate limits	Applied to login and selected high-risk endpoints.	Prevent brute force/abuse.
Authorization	Applied to protected resources.	Prevent horizontal/vertical privilege escalation.
Input validation	Applied to every request DTO.	Prevent malformed/unsafe input.

Data Components

audit_logs, security monitoring infrastructure

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

23. Frontend Feature Integration

Purpose & Functional Requirements

- Expose predictable DTOs and error codes.
- Provide loading-friendly paginated APIs.
- Return server-generated identifiers.
- Make mutation results explicit.
- Keep business-rule decisions on the backend.

API Contract

Operation	Endpoint / Mechanism	Purpose
API Contract	OpenAPI/REST DTOs	Shared contract between frontend and backend.
Errors	Typed code/message/requestId	Consistent frontend error handling.

Data Components

API DTOs and shared domain contracts

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

24. Testing & Quality

Purpose & Functional Requirements

- Unit-test business rules.
- Integration-test database constraints and transactions.
- E2E-test critical volunteer/coordinator/admin journeys.
- Test authorization boundaries and invalid inputs.
- Run load tests for peak registration/attendance/reporting workflows.

API Contract

Operation	Endpoint / Mechanism	Purpose
Unit	Domain services	Fast deterministic logic tests.
Integration	Repositories + PostgreSQL	Persistence and transaction behavior.
E2E	HTTP API	End-to-end business workflows.

Data Components

Test fixtures/factories; isolated test database

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.

- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

25. Backup, Recovery & Data Retention

Purpose & Functional Requirements

- Back up PostgreSQL according to institutional policy.
- Test restoration rather than assuming backups are usable.
- Define retention for reports, documents and audit logs.
- Archive rather than destroy historical institutional records where required.

API Contract

Operation	Endpoint / Mechanism	Purpose
Backup	Database backup process	Periodic encrypted backup.
Restore Test	Controlled environment	Verify recovery procedure.
Retention	Per data category	Apply documented retention policy.

Data Components

PostgreSQL, Object storage, audit_logs

Frontend Implementation

- Create a dedicated feature folder with page components, reusable UI components, API hooks and validation schemas.
- Implement loading, empty, success, validation-error, forbidden and server-error states.
- Use typed API response models and mutation/query hooks.
- Keep filters/pagination in URL state where the state should be shareable.

Backend Implementation

- Create a dedicated backend module with controller, application service, DTOs, domain rules and repository/data-access layer.
- Validate input at the transport boundary and enforce business invariants in the application/domain layer.
- Apply authentication and authorization before sensitive operations.
- Use database constraints and transactions for race-sensitive invariants.
- Emit audit events for sensitive administrative changes.

Acceptance Tests

- Authorized user can complete the primary workflow.
- Unauthorized user receives a permission-denied response and cannot bypass it by modifying frontend state.
- Invalid input is rejected with a structured validation error.
- Duplicate submissions do not create duplicate institutional records.
- Boundary conditions are covered by automated tests.

26. Cross-Feature End-to-End Workflows

Volunteer Lifecycle

1. User account created/activated.
2. Volunteer profile created and assigned to an NSS unit.
3. Volunteer views eligible events.
4. Volunteer registers for an event.
5. Volunteer attends through the authorized attendance workflow.
6. Verified attendance contributes to service-hour records.
7. Volunteer views achievements/certificate eligibility.

Event Lifecycle

8. Coordinator creates draft event.
9. Coordinator validates date, unit, venue, eligibility and capacity.
10. Event is published.
11. Eligible volunteers register.
12. Registration closes.
13. Attendance session opens.
14. Attendance is captured and verified.
15. Activity report is submitted/reviewed.
16. Event becomes completed and can be included in reports.

Administrative Lifecycle

17. Administrator manages users and roles.
18. Programme Officer manages assigned unit operations.
19. Coordinator reviews attendance/reports.
20. Authorized adjustments are audited.
21. Reports are generated with permission-scoped data.

27. Global API Error Codes

Code	Meaning
AUTHENTICATION_REQUIRED	No valid authenticated session.
FORBIDDEN	Authenticated user lacks required permission.
VALIDATION_ERROR	Request failed schema/business input validation.
RESOURCE_NOT_FOUND	Requested resource does not exist or is not visible.
CONFLICT	Operation conflicts with current resource state.
DUPLICATE_REGISTRATION	Volunteer already registered.
REGISTRATION_CLOSED	Event registration is not open.
EVENT_CAPACITY_REACHED	No registration capacity remains.
ATTENDANCE_SESSION_EXPIRED	Attendance session is no longer valid.
DUPLICATE_ATTENDANCE	Attendance already exists.
REPORT_NOT_READY	Asynchronous report is still processing.
RATE_LIMITED	Too many requests; retry according to server policy.
INTERNAL_ERROR	Unexpected server failure; details remain server-side.

28. Global Security Rules

- Frontend visibility is not authorization.
- Every API request must be treated as untrusted input.

- Every object access must be checked for both identity and scope.
- Never expose passwords, tokens, secret keys or internal stack traces.
- Use database constraints for uniqueness and referential integrity.
- Use transactions for multi-step state changes that must be atomic.
- Do not use permanent QR tokens as authoritative attendance credentials.
- Do not trust client-provided service-hour totals, roles, timestamps or approval state.
- Audit administrative and integrity-sensitive changes.

29. Global Testing Matrix

Area	Minimum Tests
Authentication	Valid login, invalid login, expired session, rate limit.
Authorization	Volunteer vs officer vs coordinator vs admin access.
Events	Lifecycle transitions, invalid dates, capacity.
Registration	Duplicate registration, capacity race, closed registration.
Attendance	Expired token, duplicate check-in, unauthorized check-in, correction.
Service Hours	Verified total, negative value rejection, manual adjustment approval.
Reports	Permission scoping, empty result, large export, failed job.
Documents	Invalid type/size, unauthorized download, expired access.
Audit	Sensitive operations create correct audit entries.
Database	Foreign keys, unique constraints, transaction rollback.

30. Implementation Order

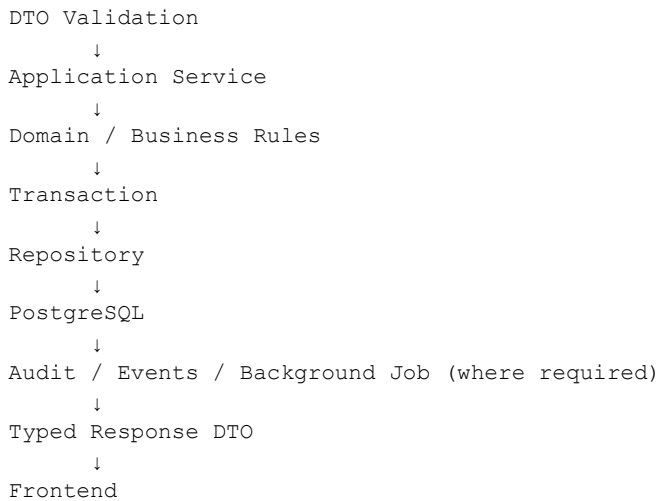
22. Foundation and database migrations.
23. Authentication, users, roles and authorization.
24. Volunteer and NSS unit management.
25. Events and event registration.
26. Attendance and QR/check-in integrity.
27. Service-hour ledger.
28. Activity reports and documents.
29. Announcements and notifications.
30. Achievements and certificates.
31. Dashboards and reports.
32. Audit, security hardening and observability.
33. Automated testing, load testing, backup/recovery and production deployment.

31. Final Feature Architecture

```

Frontend Feature
  ↓
Typed API Hook
  ↓
REST Controller
  ↓
Auth + Permission Guard
  ↓

```



This document should be used alongside the SRS, frontend implementation document and backend architecture document. Each feature should be implemented as a vertical slice: UI + API contract + backend service + persistence + authorization + tests.

32. Trade-Off Summary

Memory: Feature-oriented vertical slices and server-side pagination prevent excessive client memory usage; asynchronous jobs isolate large report/document workloads.

Latency: Indexed PostgreSQL queries, transactional writes, bounded API payloads and selective caching keep interactive workflows responsive.

Operational Overhead: A modular monolith keeps feature boundaries strong without microservice complexity; queues, object storage, observability and audit infrastructure add overhead but support reliable institutional operation.

End of Feature-Wise Implementation Specification — Version 1.0